

AVALIAÇÃO DE VALIDAÇÃO DE CONHECIMENTOS

Unidade Curricular: Programação para a Internet 2 (ProgWeb 2)

Curso Superior de Tecnologia em Sistemas para a Internet — IFSC Câmpus Garopaba

[GABARITO OFICIAL COMENTADO & RUBRICAS DE PONTUAÇÃO]

Estudante: GABARITO OFICIAL DO PROFESSOR (COM RUBRICAS DE CORREÇÃO) _____

Data de Aplicação: ____ / ____ / 2026

Nota Final (0 a 10,0): _____

Vali-

dação: [☐] Aprovado [☐] Reprovado

Instruções Gerais

- A avaliação é individual e sem consulta a materiais externos ou dispositivos eletrônicos.
- Esta prova é composta por **15 questões** divididas em duas seções:
 - **Parte 1 (Questões Dissertativas):** 3 questões analíticas (valor: 1,5 ponto cada, total: 4,5 pontos).
 - **Parte 2 (Questões Objetivas):** 12 questões de múltipla escolha ou V/F (valor: 0,45 ponto cada, total: 5,5 pontos + 0,1 ponto de arredondamento institucional = 10,0 pontos).
- Responda às questões dissertativas com clareza, utilizando terminologia técnica adequada.
- Utilize caneta esferográfica de tinta azul ou preta. Respostas a lápis não dão direito à revisão.

Parte 1: Questões Dissertativas (Valor: 4,5 pontos)

Questão 1: Arquitetura Modular no Frontend (ES6 Modules) [Valor: 1,5 ponto]

No desenvolvimento web moderno no lado do cliente, a prática tradicional de concentrar todo o código JavaScript em um script monolítico único (`app.js`) foi superada pela adoção de **ES6 Modules** (`import` e `export`) com o ponto de entrada declarado via `<script type="module" src="...">`.

Com base nos conceitos trabalhados na disciplina:

- Explique por que a separação de responsabilidades em camadas especializadas (*Services*, *Views*, *Utils* e *Orquestrador/main.js*) é preferível a um arquivo único, citando ao menos dois benefícios diretos de engenharia de software (ex: manutenibilidade, testabilidade, escopo léxico).
- Diferencie a responsabilidade estrita da camada de **Serviços** (*Services*) em relação à camada de **Apresentação** (*Views*), esclarecendo por que um arquivo de serviço não deve manipular diretamente nós da árvore DOM.

[Critério de Correção / Resposta Esperada:]

- Item (a) [0,75 pt]:** O estudante deve explicar que a arquitetura modular elimina o problema do escopo global poluído (cada módulo ES6 tem seu próprio escopo léxico isolado, evitando colisões de variáveis). Entre os benefícios esperados: facilidade de manutenção (localização rápida de erros), reutilização de código (módulos utilitários podem ser importados em várias páginas), desacoplamento para testes automatizados unitários e legibilidade de equipe.
- Item (b) [0,75 pt]:** A camada de **Serviços** (*Services*) é responsável estritamente pelas regras de negócio, persistência (leitura e gravação no `localStorage` ou chamadas HTTP via `fetch`) e tratamento de dados brutos. A camada de **Visão** (*Views*) dedica-se exclusivamente à manipulação do DOM, renderização de templates/cards, aplicação de classes de estilo (Bootstrap 5) e disparos de notificações (Toasts/Modais). Um serviço não deve tocar no DOM para que possa ser testado de forma pura e para que a interface possa ser trocada ou reformulada sem alterar a lógica de persistência e comunicação da aplicação.

Questão 2: Assincronismo, Ciclo de Vida da Rede e a Sintaxe async/await [Valor: 1,5 ponto]

No JavaScript para navegadores, operações de entrada e saída em rede (como o consumo de uma API RESTful externa via Fetch API) são tratadas de forma **assíncrona e não-bloqueante**.

Responda fundamentadamente aos seguintes itens:

- O que aconteceria com a experiência do usuário e com o comportamento da página web se uma requisição de rede que demora 3 segundos fosse executada de maneira estritamente **síncrona (bloqueante)** na thread principal da interface?
- Explique como as palavras-chave **async** e **await** atuam em conjunto com Promises no JavaScript, e explique a importância de envolver essas chamadas em uma estrutura **try/catch** para a resiliência do sistema e tratamento de falhas.

[Critério de Correção / Resposta Esperada:]

- Item (a) [0,75 pt]:** Como o JavaScript no navegador opera com um modelo de concorrência baseado em *Event Loop* em uma thread única de renderização (UI Thread), uma operação bloqueante de 3 segundos congelaria totalmente a interface gráfica. O usuário não conseguiria clicar em botões, digitar em formulários, rolar a página ou visualizar animações/spinners, gerando a percepção de que a página travou ("Not Responding"). O assincronismo delega a espera da rede para as APIs do navegador em segundo plano, mantendo a interface fluida e responsiva.
- Item (b) [0,75 pt]:** A palavra-chave **async** transforma uma função para que ela retorne implicitamente uma Promise. A palavra-chave **await** só pode ser usada dentro de funções **async** e pausa a execução daquela função específica até que a Promise seja resolvida (sem travar a thread principal). O bloco **try/catch** é essencial porque qualquer rejeição de Promise (como falha física de conexão, timeout, servidor offline ou erro de DNS) gera uma exceção assíncrona; sem o **catch**, a exceção fica não tratada (*Uncaught Promise Rejection*), interrompendo o fluxo da aplicação e impedindo o disparo de mensagens de aviso (Toasts de erro) ou o chaveamento para rotinas de contingência (fallback).



Questão 3: Protocolo HTTP, Segurança de Rede (CORS) & Arquitetura Híbrida [Valor: 1,5 ponto]

Considere o cenário em que desenvolvemos uma aplicação web cujo frontend está hospedado localmente no Live Server (porta 5500) e consome uma API REST construída em Python com o micro-framework Flask rodando na porta 5000.

Analise a integração e responda:

- O que significa a sigla **CORS** (*Cross-Origin Resource Sharing*), por que o navegador bloqueia requisições entre origens diferentes por padrão, e qual componente no servidor Flask é responsável por enviar os cabeçalhos de liberação?
- Em uma aplicação profissional com deploy no **GitHub Pages** (que hospeda apenas arquivos estáticos e não executa servidores Python), explique como deve ser arquitetado o serviço de dados (`vitrineService.js`) para garantir uma **contingência resiliente (fallback)** entre o banco de dados remoto da API e o armazenamento local do navegador (`localStorage`).

[Critério de Correção / Resposta Esperada:]

- Item (a) [0,75 pt]:** CORS significa *Cross-Origin Resource Sharing* (Compartilhamento de Recursos de Origem Cruzada). O navegador bloqueia por padrão em decorrência da Política de Mesma Origem (*Same-Origin Policy* - SOP), um mecanismo fundamental de segurança que impede que scripts maliciosos injetados em um site leiam dados privados de APIs de outro domínio sem autorização. No Flask, utilizamos a biblioteca/extensão `flask-cors` (função `CORS(app)`), que anexa os cabeçalhos HTTP necessários (como `Access-Control-Allow-Origin: *`) nas respostas da API, autorizando explicitamente o navegador a liberar os dados para o frontend.
- Item (b) [0,75 pt]:** Para garantir resiliência, a camada de serviço implementa uma estratégia híbrida: dentro da função `async`, o código tenta primeiro realizar o `fetch()` para a URL da API remota. Se a requisição for bem-sucedida, ele converte o JSON, atualiza uma cópia em cache no `localStorage` e retorna os dados do banco relacional. Caso a requisição falhe (seja por timeout, servidor Flask desligado ou por estar publicado no GitHub Pages estático sem o backend ativo), o fluxo é capturado no bloco `catch`; nele, a aplicação aciona de forma transparente a leitura da cópia armazenada no `localStorage`, retornando os dados locais e sinalizando na interface que o sistema está em modo offline/contingência, evitando que a página quebre para o usuário.

Parte 2: Questões Objetivas (Valor: 5,5 pontos)

Questão 4: Declaração de Módulos no HTML5

Para que o navegador interprete um script JavaScript reconhecendo as palavras-chave `import` e `export` e isolando seu escopo em nível de arquivo, qual atributo deve ser obrigatoriamente incluído na tag `<script>`?

- a) `<script rel="module"src="js/main.js">`
- b) `<script async="true"src="js/main.js">`
- c) **`<script type="module"src="js/main.js">`**
- d) `<script language="javascript6"src="js/main.js">`

[Critério de Correção / Resposta Esperada:] A diretiva `type="module"` ativa o carregamento estrito de módulos ES6 nativos no navegador.

Questão 5: Sistema de Grid e Responsividade do Bootstrap 5

No sistema de Grid de 12 colunas do Bootstrap 5, considere o seguinte elemento aplicado em um card da vitrine:

```
<div class="col-12 col-md-6 col-lg-4">
```

Assinale a alternativa que descreve **corretamente** o comportamento desse elemento em diferentes dispositivos:

- a) O card ocupa 4 colunas em celulares, 6 colunas em tablets e 12 colunas em monitores desktop.
- b) **O card ocupa a largura total da linha (12 colunas / 1 card por linha) em celulares, metade da largura (6 colunas / 2 cards por linha) a partir de telas médias (md) e um terço da largura (4 colunas / 3 cards por linha) em telas grandes (lg).**
- c) O elemento divide a tela em 12 colunas estáticas em qualquer tamanho de monitor.
- d) O elemento quebra para a linha seguinte apenas se a resolução for menor que 320 pixels.

[Critério de Correção / Resposta Esperada:] O Grid do Bootstrap é mobile-first: `col-12` é o padrão base; a partir de 768px (md) assume 6 colunas e a partir de 992px (lg) assume 4 colunas.



Questão 6: Persistência no Navegador (Web Storage API)

A respeito das diferenças entre `localStorage` e `sessionStorage`, assinale a afirmação **CORRETA**:

- a) O `sessionStorage` mantém os dados gravados mesmo após o fechamento completo do navegador e reinicialização do sistema operacional.
- b) O `localStorage` tem seus dados apagados automaticamente assim que o usuário fecha a aba correspondente.
- c) **O `localStorage` persiste os dados sem data de expiração, mantendo-os disponíveis mesmo se o navegador for reiniciado, enquanto o `sessionStorage` é destruído ao fechar a aba/sessão da página.**
- d) Ambos transmitem automaticamente todos os seus dados ao servidor em todas as requisições HTTP, da mesma forma que os Cookies.

[Critério de Correção / Resposta Esperada:] O `localStorage` é não-volátil (permanece até ser limpo explicitamente); `sessionStorage` vive apenas no ciclo de vida daquela aba.

Questão 7: Serialização JSON para Armazenamento Local

Como o `localStorage` aceita exclusivamente o armazenamento de valores no formato de texto (*string*), quais métodos nativos do objeto JSON devem ser empregados, respectivamente, para **salvar um array de objetos** e para **restaurá-lo como objeto manipulável em memória**?

- a) `JSON.toText()` e `JSON.toObject()`
- b) **`JSON.stringify()` para salvar e `JSON.parse()` para recuperar.**
- c) `JSON.encode()` para salvar e `JSON.decode()` para recuperar.
- d) `JSON.serialize()` para salvar e `JSON.unserialize()` para recuperar.

[Critério de Correção / Resposta Esperada:] `JSON.stringify()` converte estruturas de dados JavaScript em texto JSON; `JSON.parse()` reconverte texto JSON em objetos/arrays reais.



Questão 8: Componentes de Feedback no Bootstrap 5

Sobre a utilização dos componentes **Modal** e **Toast** do Bootstrap 5, assinale a alternativa **CORRETA**:

- a) O *Toast* é um diálogo sobreposto bloqueante com fundo escuro (*backdrop*) utilizado para preenchimento de formulários extensos.
- b) O *Modal* só pode ser criado dinamicamente via JavaScript puro, sendo proibida a declaração em marcação HTML.
- c) **O *Modal* é uma janela de foco crítico que sobrepõe a tela com *backdrop* bloqueante, enquanto o *Toast* é uma notificação flutuante leve, temporizada e não-bloqueante utilizada para confirmar operações.**
- d) Ambos os componentes recarregam a página inteira (*Page Reload*) no momento em que são abertos.

[Critério de Correção / Resposta Esperada:] Modais bloqueiam a navegação para exigir foco exclusivo (ex: formulário de cadastro); Toasts notificam discretamente (ex: "Serviço salvo!").

Questão 9: Protocolo HTTP e Códigos de Status (Status Codes)

Ao interagir com uma API RESTful, o servidor responde com códigos numéricos que indicam o resultado da operação. Relacione a operação ao seu código de status HTTP padronizado esperado:

- I. Criação bem-sucedida de um novo serviço no banco via POST.
- II. Leitura bem-sucedida da listagem de serviços via GET.
- III. Tentativa de buscar um serviço com identificador inexistente no banco.
- IV. Erro fatal de execução no código interno do servidor Python.

A sequência correta de códigos correspondentes é:

- a) I - 200; II - 201; III - 500; IV - 404
- b) **I - 201; II - 200; III - 404; IV - 500**
- c) I - 204; II - 200; III - 400; IV - 502
- d) I - 201; II - 204; III - 403; IV - 400

[Critério de Correção / Resposta Esperada:] 201 Created (criação com sucesso), 200 OK (leitura/operação bem-sucedida), 404 Not Found (recurso não encontrado), 500 Internal Server Error.

Questão 10: Envio de Dados com a Fetch API

Para enviar um novo registro em formato JSON para uma rota de backend `/api/servicos` via método POST, qual a configuração correta dos parâmetros da função `fetch()`?

- a) `fetch('/api/servicos', { method: 'GET', data: dados });`
- b) `fetch('/api/servicos', { method: 'POST', headers: { 'Content-Type': 'application/json' }, body: JSON.stringify(dados) });`
- c) `fetch('/api/servicos', { type: 'POST', content: dados.toString() });`
- d) `fetch('/api/servicos?action=post&data=' + dados);`

[Critério de Correção / Resposta Esperada:] O verbo POST exige cabeçalho Content-Type: application/json e payload serializado no corpo (body) via JSON.stringify.

Questão 11: Validação Visual no Cliente (Bootstrap 5)

(Verdadeiro ou Falso) Ao aplicar a classe `.was-validated` no elemento `<form>` do Bootstrap 5 após interceptar o evento submit, os navegadores acionam as pseudo-classes CSS `:valid` e `:invalid`, exibindo imediatamente os estilos visuais de borda verde ou vermelha e as mensagens contidas nos contêineres `.valid-feedback` e `.invalid-feedback`.

- (X) Verdadeiro
- () Falso

[Critério de Correção / Resposta Esperada:] Verdadeiro. O Bootstrap 5 utiliza a classe `.was-validated` no form para ativar as pseudo-classes nativas do CSS em todos os inputs.



Questão 12: Arquitetura de Single Page Applications (SPAs)

Qual das seguintes características define fundamentalmente uma aplicação construída sob a arquitetura de **Single Page Application (SPA)**?

- a) O site possui apenas uma página de texto e proíbe a exibição de mais de uma seção ou imagem.
- b) O backend renderiza todo o HTML no servidor e recarrega a página inteira a cada link clicado pelo usuário.
- c) **A navegação e a troca de telas ocorrem no lado do cliente via JavaScript, manipulando dinamicamente o DOM sem a necessidade de recarregar a página inteira pelo navegador (no full page refresh).**
- d) As páginas de uma SPA não utilizam folhas de estilo CSS nem eventos de formulários.

[Critério de Correção / Resposta Esperada:] SPAs carregam o esqueleto inicial e alteram as visões e dados dinamicamente sem recarregamento da página inteira pelo navegador.

Questão 13: Delegação de Eventos no DOM

Ao renderizar dinamicamente dezenas de cards de serviços contendo botões de exclusão (`class="btn-excluir"`), a técnica mais eficiente e performática consiste em registrar um único ouvinte de evento de clique no contêiner pai (`vitrineContainer`) e identificar o botão clicado através de:

- a) `event.stopPropagation()`
- b) **`event.target.closest('.btn-excluir')`**
- c) `document.querySelectorAll('.btn-excluir').forEach(...)`
- d) `window.location.reload()`

[Critério de Correção / Resposta Esperada:] A técnica de delegação de eventos utiliza `closest()` para detectar se o clique ocorreu no botão desejado ou em seus elementos filhos (ex: ícone `<i>`).



Questão 14: Ecossistemas Web Modernos e Ferramentas de Build (Vite)

Em projetos frontend profissionais, ferramentas modernas de automação e empacotamento como o **Vite** com **npm / Node.js** são utilizadas prioritariamente para:

- a) Substituir o navegador web na renderização gráfica da página.
- b) **Oferecer um servidor de desenvolvimento ultra-rápido com Hot Module Replacement (HMR) e compilar/minificar o código final em pacotes otimizados para produção.**
- c) Configurar roteadores de internet física e cabeamento de rede.
- d) Proibir o uso de bibliotecas de terceiros no projeto.

[Critério de Correção / Resposta Esperada:] Vite é um bundler moderno que acelera o desenvolvimento local com HMR e empacota bundles altamente otimizados para produção.

Questão 15: Integração Frontend-Backend com Flask e SQLite

(Verdadeiro ou Falso) O banco de dados **SQLite** utilizado no microbackend em Python Flask armazena todas as tabelas e registros em um único arquivo local no disco (como `vitrine.db`), dispensando a instalação e configuração de servidores de banco dedicados pesados (como MySQL ou PostgreSQL), o que o torna ideal para práticas pedagógicas de laboratório e protótipos ágeis.

- ☒ Verdadeiro
- ☐ Falso

[Critério de Correção / Resposta Esperada:] Verdadeiro. O SQLite é um banco de dados relacional embutido e baseado em arquivo único, que não requer servidor ou porta de rede própria.

— Boa Prova! —